3D Gaussian Splatting Reconstruction

Team Gauss
Jackson Culbreth (Team Lead)
Joshua Bowman (Scrum Master)
Jackson Kelly (Developer)
Nicholas Desmornes (Developer)

CIS 4914 Senior Project

Advisor: Dr. Alireza Entezari (entezari@ufl.edu)

Presentation Link: https://www.youtube.com/watch?v=pnE956J9BeQ

Table of Contents

Keywords

Gaussian Splatting, 3D Reconstruction, 3D Computer Graphics, Structure-from-Motion, COLMAP, VGGT, FasterGS, Opensplat, novel view synthesis, CUDA, PyTorch

Project Overview - 3D Gaussian Reconstruction

In recent years, researchers have developed an alternative method for 3D scene reconstruction. Instead of typical techniques using meshes, this approach uses thousands or millions of tiny gaussians and a machine learning optimization process to create a 3D scene from 2D images. These gaussians are tiny blobs that contain information about the point's position, color, opacity, shape, and size. The optimization process attempts to reduce photometric error, which is the difference between the pixels in the 2D images, and the corresponding points on the 3D reconstruction. Each iteration updates the gaussians in order to accomplish this. The end result is a detailed 3D construction of the original scene.

Due to the novelty of this technique, large amounts of the work being done is modular, and the technology is fairly inaccessible. It would take significant time and effort for a researcher or interested party to access a working implementation. Therefore, our project, 3D Gaussian Reconstruction, aims to provide an end-to-end gaussian splatting pipeline, allowing a user to upload original video and receive a 3D render of their exact scene. We aimed to implement a complete pipeline based on existing gaussian splatting techniques, as opposed to developing a novel algorithm. This pipeline includes data ingestion, preprocessing, Structure-from-Motion,

initialization and optimization of gaussians, and scene rendering. This provides a complete implementation of gaussian splatting, increasing access to the technology.

Our team consisted of Jackson Culbreth as team lead, Joshua Bowman as scrum master, and Jackson Kelly and Nicholas Desmornes as developers. The team lead was responsible for overseeing project progress and ensuring milestones were met, coordinating assignments and deliverables, assisting developers, and serving as main point of contact with the project advisor. The scrum master was tasked with facilitating meetings and sprint planning, removing blockers and ensuring team accountability, and maintaining task board and sprint documentation. The development team was responsible for implementing the gaussian splatting pipeline and supporting services, designing the UI, and collaborating on performance optimization.

Deliverables

Our original project specification consisted of building data ingestion, employing various preprocessing to optimize data for training, using COLMAP / OpenSfM for sparse reconstruction, initializing and optimizing gaussian splats, developing renderer and interactive viewer, and completing a comprehensive application UI. These were all successfully completed. Our data ingestion is handled by a FastAPI endpoint that handles upload from the frontend, and then a python script using OpenCV to save the file and extract frames for the pipeline. Our preprocessing included filtering for blur, using the variance of the image, and duplicate frames, using a mean absolute difference calculation. Additionally we had controls for resizing and

frame rate. We used OpenSfM to generate our sparse reconstruction, extracting shared features among images and estimating camera positioning to get a 3D point cloud. Our gaussian splatting was implemented using OpenSplat, handling our key initialization and optimization of the gaussians. Our renderer was implemented as a React component using React Three Fiber and Drei. It loads the splat file generated by Opensplat from the backend and displays it using WebGL. Our UI was built with React and provides a complete and intuitive user experience.

Our project status is largely completed. All the goals and features we originally planned have been successfully implemented and are fully working. Any remaining work would focus on testing different approaches to splatting in order to contribute to the scientific developments in the field, which falls outside of our original project scope.

One limitation of our implementation is the use of OpenSplat. We used a C++ binary to handle the optimization process of our gaussian splatting, this does mean that in order to run our project successfully, a user would need to have OpenSplat successfully set up on their machine. This can be quite a convoluted process, and thus could be beneficial to switch to a PyTorch centered implementation if the project were to be deployed in order to reduce setup complexity.

Technical Details

The project's code base was maintained in a github repository. The main branch was intended to be stable at all times. Each developer worked individually on their own branches, opening a pull request to the main branch once they were completed with their task. All pull

requests were reviewed by someone other than the submitter in order to be merged to the main branch.

Our dependencies included OpenSfM for reconstruction, Opensplat as a compiled executable for gaussian splatting, FastAPI and OpenCV for backend processing, and React Three Fiber with Three.js for rendering and visualization.

In order to deploy the project, python3 and NodeJS are required. Frontend dependencies can be installed by running “npm install” from the frontend folder. After running “npm run dev”, the project’s frontend will be available at `http://localhost:5173` in a browser. For the backend, a user needs to create and activate a virtual environment in the backend folder. This can be done by running “python3 -m venv venv” and then “source venv/bin/activate” or Mac or Linux versus “venv\Scripts\activate” on Windows. All backend dependencies can then be installed with “pip install requirements.txt”. The server should then be run with: “uvicorn main:app --reload”. Opensplat installation instructions can be found from the opensplat repository, and the resulting binaries should be placed in the backend/binaries folder.

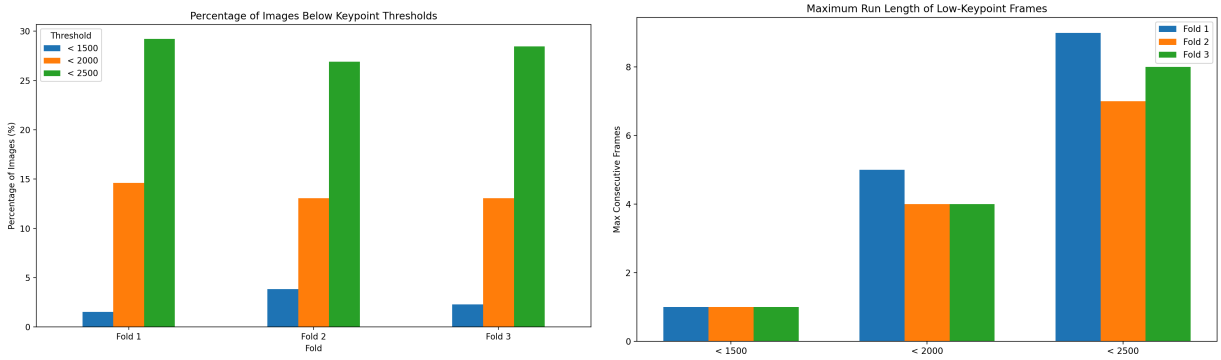
Our project architecture follows a modular full stack design. This consists of separate layers for the frontend, API, and processing. This is accomplished with a React frontend for user interaction, FastAPI backend for uploading videos and facilitating the pipeline, and a modular processing pipeline including video ingestion, preprocessing, structure-from-motion, gaussian splatting, and rendering. Video is uploaded by the user on the frontend, which triggers the FastAPI endpoint to save the video file, create a dataset directory, and start the pipeline. The

pipeline then runs through the preprocessing, structure-from-motion, and gaussian splatting, which outputs a .splat file. This file is then used by the frontend and displayed in the interactive renderer. All the storage throughout the process is done in the local file system.

Testing Information

I. SfM - Image Feature Extraction Testing

We found we could measure the quality of our datasets (e.g. resolution, consistency, trackability) by compiling data on the keypoints the SfM computation was generating. The threshold chart in Figure 1.1 tracked the percentage of images in the dataset that generated a number of keypoints beneath a certain threshold. By ensuring that <10% of images have less than 1,500 keypoints, we could be reasonably confident that our dataset did not have too many low quality images that would result in a poor point cloud. The run length chart in Figure 1.2 recorded the longest consecutive streak of low-keypoint frames. Isolated weak frames aren't concerning, but longer runs could lead to sections that are difficult for SfM to reconstruct. Spotting datasets that generate higher streaks of less than 1,500 keypoints could indicate poor reconstruction. Figure 1.3 compiles the rolling mean of keyframes over the image indices. This illustrates the quality of the feature richness across the sequence over time. Dips in the rolling mean can indicate sections where the scene is difficult to extract features.



Figures 1.1: Keypoint Thresholds, Figure 1.2: Max Low-Keypoint Run Length

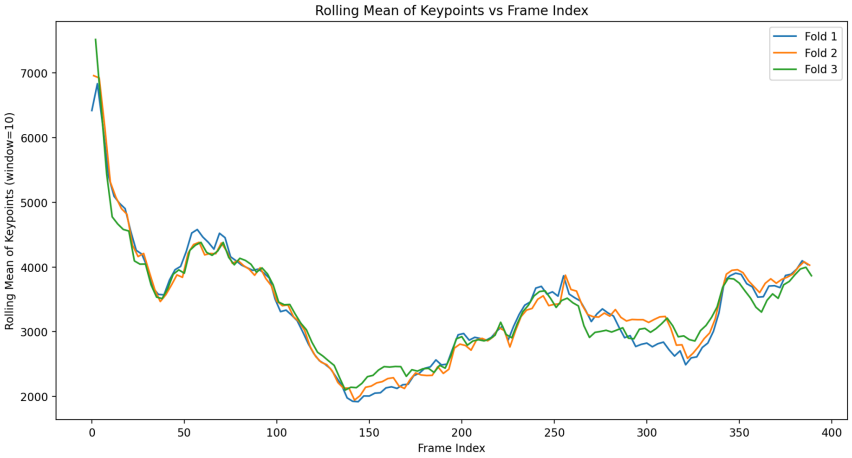


Figure 1.3: Keypoints Rolling Mean vs. Frame Index

During experimentation with various datasets with SfM, we found that capturing scenes from different viewpoints, rather than standing in the same location, had the best results. We thought it best to record the University of Florida Stadium by standing at the center of the field and rotating the camera in-place to capture the entire interior. But this resulted in numerous mathematical errors during execution and eventually resulted in a very poor initial point cloud and Gaussian splatting. This issue may be due to poor geometric verification in correspondence search. Not providing enough variability in camera positions led to computation that fell apart. In addition, we also found that datasets with blurry images, inconsistent illumination, or fewer

images overall resulted in degraded results. These findings are also supported by the increased difficulty in correspondence search with feature extraction and matching that such datasets would induce.

II. Gaussian Splatting Metrics

Various metrics could be compiled from our Gaussian splatting jobs to evaluate the underlying implementation and quality of the results. The following experiment tested an image dataset of a truck with 251 images, generating a sparse point cloud with 67,124 initial points. The loss function (defined in Equation 4, discussed in User Documentation) is plotted over iterations of the training (Figure 2). The data is plausible as it follows a loosely monotonically decreasing curve as it optimizes the Gaussians. We found that the spikes in loss every few thousand iterations were caused by a design decision by 3DGS to reset Gaussian opacity (α) to 0 every 3,000 iterations (stopped at 15,000 iterations). The purpose is to address a problem where the optimization can get stuck with floaters (Gaussians with very low opacity) close to the input cameras, resulting in an unjustified increase in Gaussian density [2]. Resets allow the algorithm to increase α for necessary Gaussians whilst culling will remove near invisible opacity Gaussians below a certain threshold.

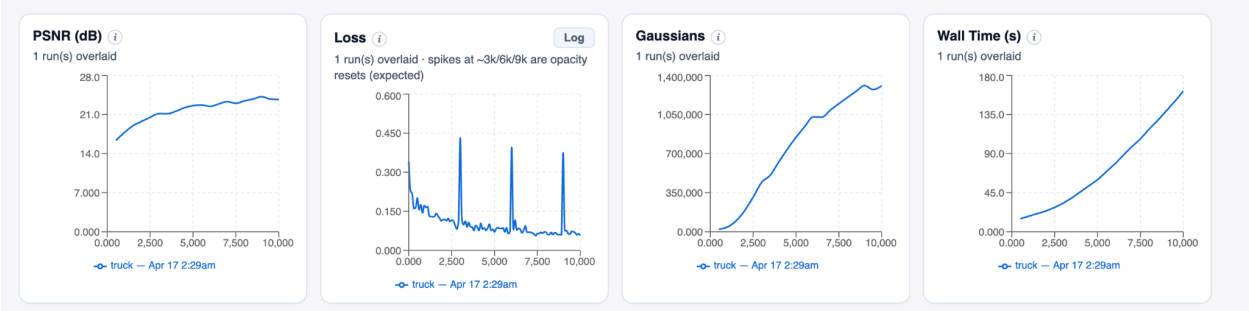


Figure 2: Gaussian Splatting Metrics - Truck dataset

Other metrics compiled to measure performance include the Gaussian count, Peak-to-Signal-Ratio (PSNR), and wall time (runtime). Our experiments found that the most impactful factor for good Gaussian splatting results was initializing with a high quality point cloud, rather than total training iterations. In our case, we tested the SfM algorithm choice between COLMAP and VGGT (a deep learning implementation). COLMAP outperformed VGGT by 1.90 to 6.81 dB across three scenes (Figure 3). Tripling training iterations improved PSNR by roughly 2.5 dB while switching to a better SfM method improved it by over 6 dB. These metrics and results serve as a good benchmark to test future advancements in Gaussian splatting solutions, which we'll explore in the Transition Plan/Next Steps section.

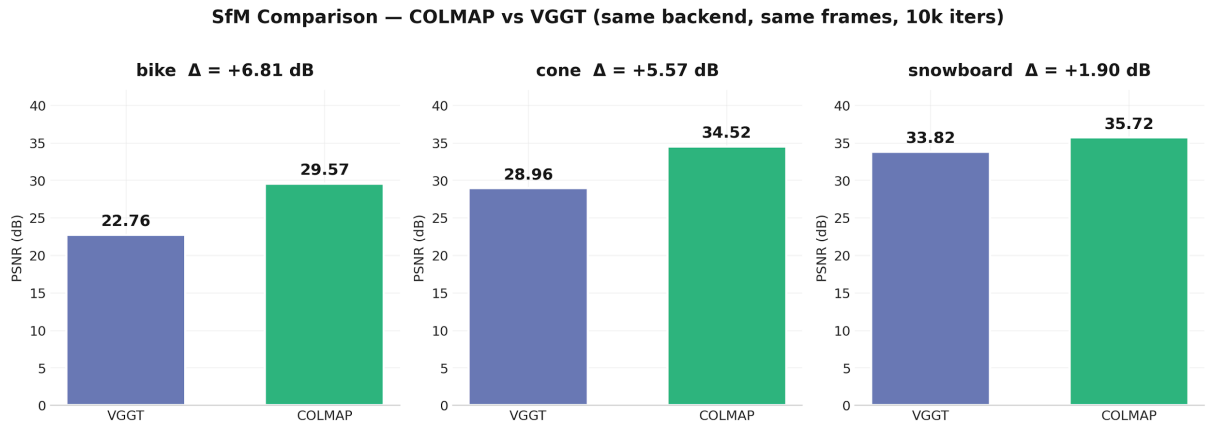


Figure 3: PSNR - COLMAP vs VGGT SfM

III. Faster-GS Integration

We integrated the Faster-GS CUDA backend into our pipeline via the author's reference integration fork of the original Inria 3DGS codebase [7]. The full Faster-GS framework is

available as a native NeRFICG-based implementation. We chose the Inria integration fork to minimize surface area changes in our pipeline. Our HiPerGator environment constrained us to L4 (sm_89) and B200 (sm_100) GPUs. The Faster-GS custom rasterizer was tuned on Ampere (sm_80, ~164KB shared memory per block) and could not launch on L4 (~100KB) due to shared memory requirements, nor on B200 until the fork ships sm_100 binary support.

The Faster-GS authors optimized their custom CUDA rasterizer for shared memory budgets available on NVIDIA Ampere architecture (A100: ~164KB per block), on which they report 3-5x speedups over the baseline Inria rasterizer. Porting this kernel to Ada Lovelace (L4: ~100KB per block) would require reducing the tile size and increasing the number of blocks, which would also reduce the achievable speedup. This illustrates a general tradeoff in GPU research code.

To fix this issue, we isolated the issue to the rasterizer launch configuration. Then validated that the fused Adam optimizer component works correctly across both tested architectures, providing a partial speedup pathway. We are still testing if it gives good quality renders. So therefore fell back to the stock Inria rasterizer with stock Adam and use only fused Adam for testing.

User Documentation

I. Gaussian Splatting

i. Background & Introduction

The realm of realistic 3D environment generations with real-time rendering and navigation has had a long history of research and advancement. A subsection of this field aims to make content creation for 3D graphics easier through automatic 3D reconstruction from real-world 2D photos or video, as well as allowing free-viewpoint navigation or view synthesis. Among the primary modern-day methods to achieve this are Neural Radiance Fields (NeRFs) and 3D Gaussian Splatting (3DGS). NeRF is an algorithm that represents 3D scenes using a fully-connected neural network [1]. The major drawback with this method is that it's computationally intensive to render. Each new image of a scene will be slow to generate due to NerF's implicit representation; querying the neural network for each of the new pixels via ray passing. Gaussian splatting improves upon this issue by instead constructing a scene with 3D Gaussians and rendering via rasterization, successfully achieving high-quality real-time view synthesis [2].

Our project aims to create an application that condenses 3DGS into a user-friendly pipeline, thus making the 3DGS workflow seamless to amateur users as well as more practical for understanding. This potentially allows users to pick up the project and make independent changes to implement optimizations. The application incorporates various libraries in addition to the core pipeline, therefore it's important to learn the details behind 3D Gaussian Splatting to understand the underlying workflow of the application. The following writeup will walk through what our team has learned in conjunction with the accompanying literature and libraries utilized.

ii. Workflow Pipeline

From a high level, Gaussian Splatting is a technique that constructs scenes out of 3D gaussians from 2D images with excellent real-time rendering performance. The process of 3DGS is broken down into three major steps: Structure from Motion (SfM), Gaussian Splatting Training, and Rendering. Each of these steps will be discussed in detail with respect to the corresponding literature and libraries used in our project.

SfM & COLMAP. The input for 3DGS requires a set of 2D images or video capturing a static scene, along with data describing the camera positions and orientations and a sparse 3D point cloud (Figure 4) [2]. The camera data and sparse point cloud are generated by an algorithm called Structure-from-Motion (SfM). This input provides the parameters needed to allow the Gaussians to train and converge towards an accurate representation of the captured scene. We will elaborate on the stages of the SfM processing pipeline.

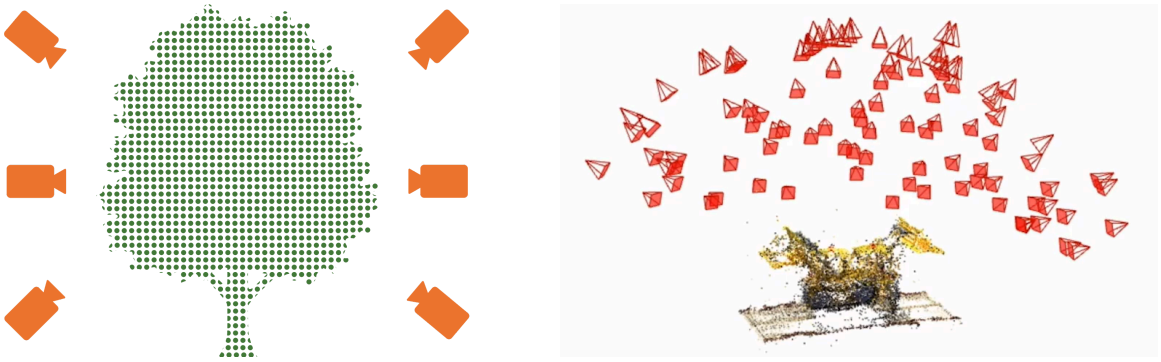


Figure 4: SfM generated camera positions and sparse point cloud (left: general representation, right: real graphic example) [10]

SfM is the process of reconstructing 3D structure from its projections into a series of overlapping images of the same object taken from different viewpoints [3]. The specific strategy used here, incremental SfM, is a sequential processing pipeline with an iterative reconstruction component (Figure 5).

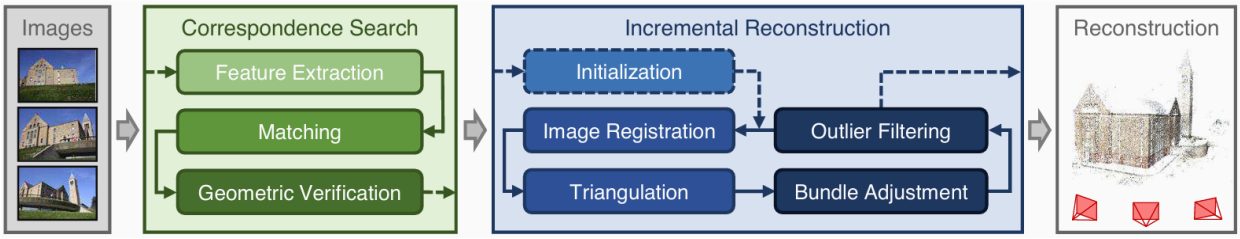


Figure 5: Incremental Structure-from-Motion pipeline source: [3]

The first stage of the pipeline is correspondence search. The term correspondence is used frequently in the context of finding relations between two views because it describes determining image points of the first view x that “correspond” to the image points of the second view x' [4]. In other words, correspondence search identifies shared points in overlapping images. Feature extraction is used to detect local features on the 2D images [3]. Algorithms such as SIFT and its derivatives are used to execute feature extraction. Next, SfM performs matching, which searches for feature correspondences between pairs of images. This will generate a set of potentially overlapping image pairs. A visual of feature extraction and matching is shown in Figure 6.

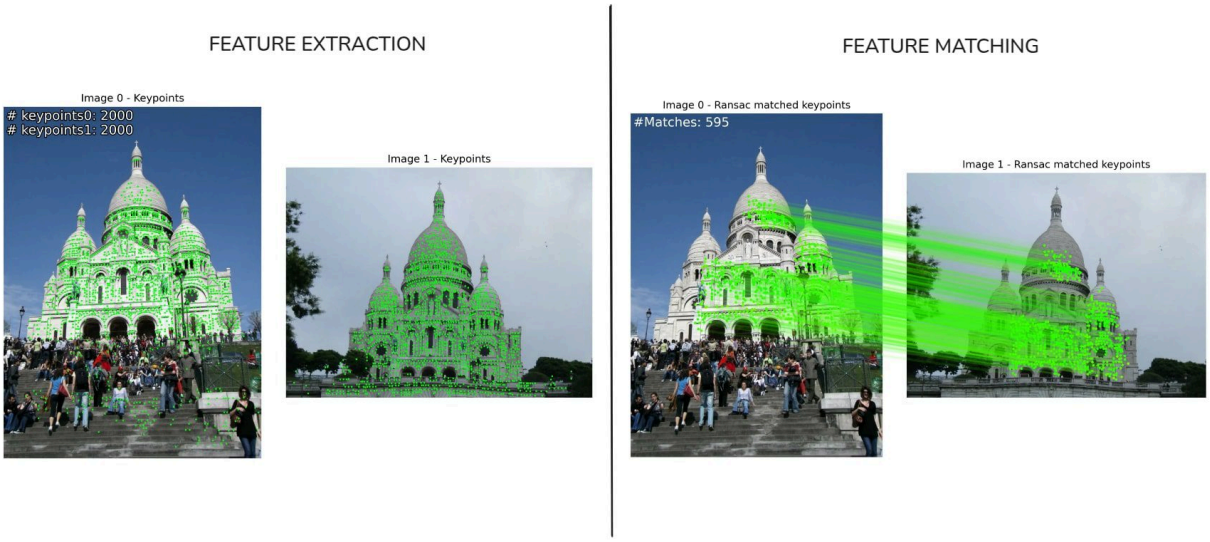


Figure 6: Feature Extraction and Feature Matching [11]

Matching is not a guarantee that corresponding features between pairs of images actually map to the correct identical scene point. Thus, geometric verification is the third step needed to verify the matches using projective geometry principles and transformations. In the SfM publication's words, "Epipolar geometry (Figure 7) describes the relation for a moving camera through the essential matrix **E** (calibrated) or fundamental matrix **F** (uncalibrated)" [3]. If a valid transformation maps a sufficient number of features between images, they are considered geometrically verified. Additional techniques such as RANSAC are required to eliminate outliers in correspondences. The output will be a set of geometrically verified image pairs, finalized as a scene graph with the images as the nodes and the verified pairs of images as the edges.

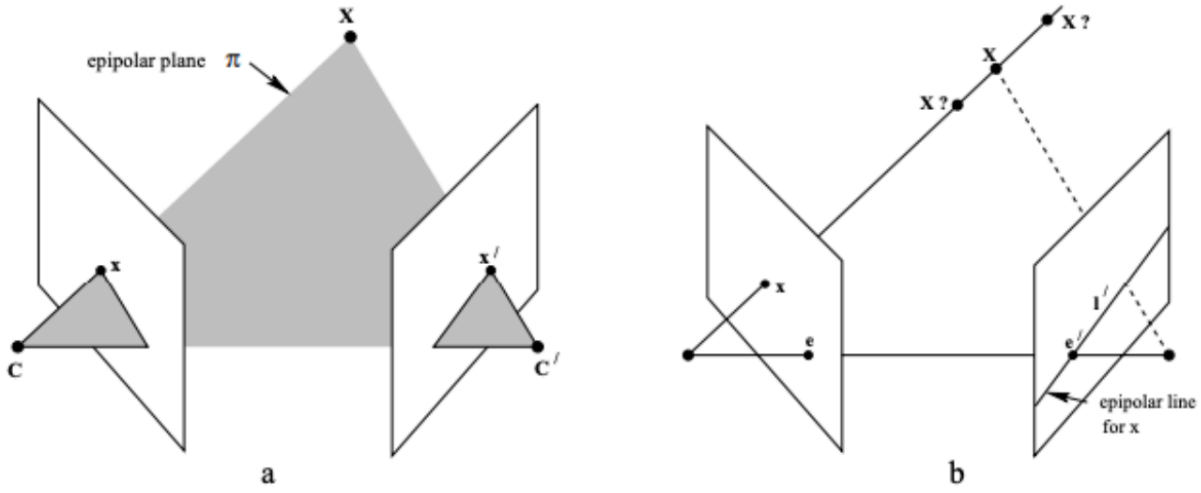


Figure 7: Point correspondence geometry (a) Points X, camera centres, and images x and x' form a common epipolar plane π. (b) Ray connecting X and first camera centre C imaged as line l' in second view. The second view's point X must lie on projected line l'. [4]

The next stage of the pipeline is incremental reconstruction. Given the scene graph of images and verified image pairs, this stage constructs the 3D point cloud. First, the procedure runs initialization by selecting an image pair that yields a strong initial 3D model. A strong initialization is important because bad initialization effects will propagate throughout the reconstruction process. Next, image registration, or the registering of new images to the current model, is performed by solving the Perspective-n-Point (PnP) problem [3]. The PnP problem estimates the camera pose and intrinsic parameters. Then, through triangulation, scene coverage is increased. A new scene point can be calculated and added to the point cloud when a different viewpoint covering the same scene is registered. Triangulation increases stability of the existing model through redundancy, enabling registration of new images by providing additional correspondences. Finally, bundle adjustment is required to further refine calculated parameters to

prevent state instability. Bundle adjustment is joint non-linear refinement that minimizes the reprojection error [3].

The SfM code has been contributed to the community as an open-source implementation named COLMAP, available at https://github.com/colmap/colmap [3]. This is the main library our project utilizes to run SfM in our pipeline. The corresponding paper on SfM is *Structure-from-Motion Revisited* by Johannes L. Schönberger and Jan-Michael Frahm. Further information about COLMAP and SfM can be found at COLMAP's tutorial page at https://colmap.github.io/tutorial.html.

Gaussian Splatting Training. Next, Gaussian splatting optimization is used to construct a dense set of 3D Gaussians to accurately represent the 3D scene for view synthesis. Gaussians are bell curves often used to describe probability P(x) (Figure 8.1). In the context of Gaussian splatting, the probability function can instead be visualized as the density or opacity of the color. When transformed into 3-dimensions, the Gaussian becomes what can be described as a 3D ellipsoid that fades around the outer edges (Figures 8.2, 8.3).

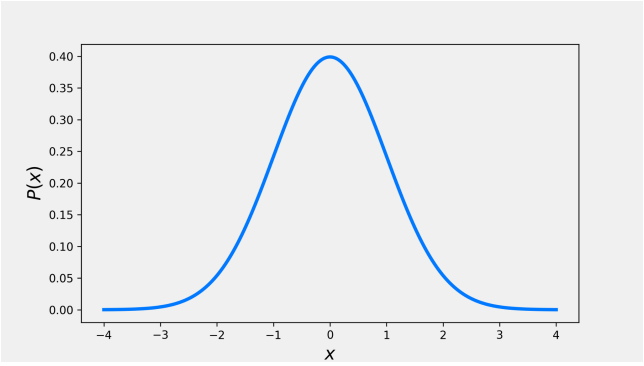
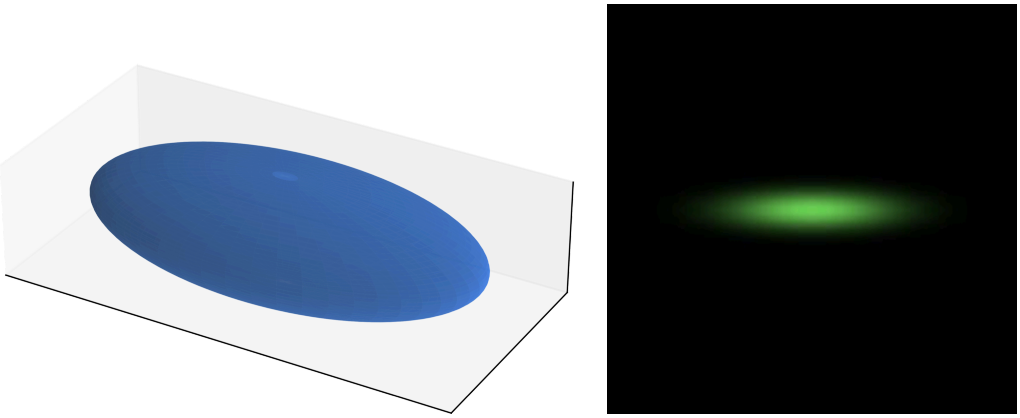


Figure 8.1: 1D Gaussian [10]



Figures 8.2, 8.3: 3D Gaussians [5, 10]

The Gaussians are defined by their position or mean, covariance matrix, and opacity α. The 3D covariance matrix Σ centered at the mean μ effectively describes their shape, size, and orientation in space. The color is represented using spherical harmonics (SH). 3D Gaussians were chosen because they are differentiable and easily projected to 2D splats, allowing for fast α-blending for rendering [2].

$$G(x) = e^{-\frac{1}{2}(x)^T\Sigma^{-1}(x)}$$

Equation 1: Gaussian Function

With these inherent properties, a flexible optimization regime is set to optimize a scene representation. Given the 2D images of the scene, the camera data, and sparse point cloud provided by SfM, the optimization process of 3D Gaussian splatting can begin (Figure 9). The sparse point cloud is used to initialize the set of 3D Gaussians. While point cloud initialization is not strictly required, it has been found to help speed up convergence and attain high-quality results [2].

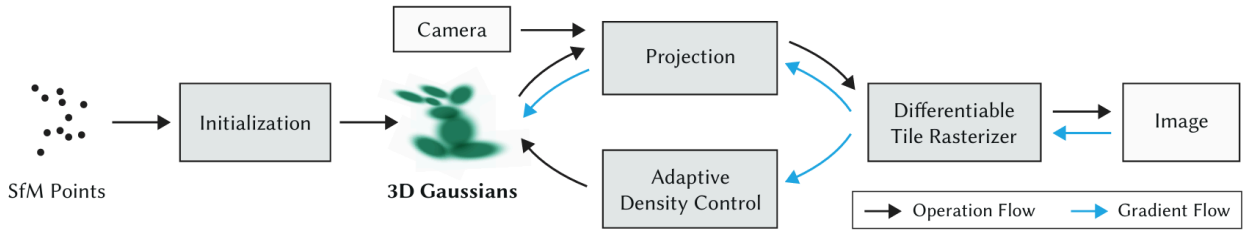


Figure 9: 3D Gaussian Optimization Flow [2]

Then the optimization of the Gaussians to accurately represent the scene is set into motion. It involves many successive iterations of rasterizing and comparison to the training dataset for disparity (rasterizing/rendering will be covered in detail in the next section). For optimization of the covariance, it is represented as a product of the rotation matrix R and the scaling matrix S (Equation 2). The color is handled through the optimization of the SH coefficients to capture the view-dependent appearance of the scene [2].

$$\sum = RSS^T R^T$$

Equation 2: Covariance Matrix for Optimization

The 3D Gaussians will be projected to 2D for rendering. The projection to image space is described as a transformation W which calculates the covariance matrix Σ' where J is the Jacobian (Equation 3).

$$\sum' = JW\sum W^T J^T$$

Equation 3: Projection Function

The loss function (Equation 4) calculated can be described as a combination of determining color similarity between the projected pixel colors and the corresponding 2D image inputs, and structural similarity which evaluates luminance, contrast, and structure. The optimizer uses Stochastic Gradient Descent techniques which utilizes GPU-accelerated frameworks and CUDA kernels [2]. An illustration of the forward and backward passes for an implementation by gsplat can be found in Appendix A.

$$L = (1 - \lambda)L_1 + \lambda L_{D-SSIM}$$

Equation 4: Loss Function

During training, the Gaussians undergo adaptive control to influence the number of Gaussians and their densities to converge towards the optimized scene representation [2]. As illustrated in Figure 10, Gaussians can be duplicated or split to better fit the 3D scene representation. In addition, Gaussians that are effectively transparent ($\alpha < \epsilon_\alpha$, where ϵ_α is a threshold) are removed.

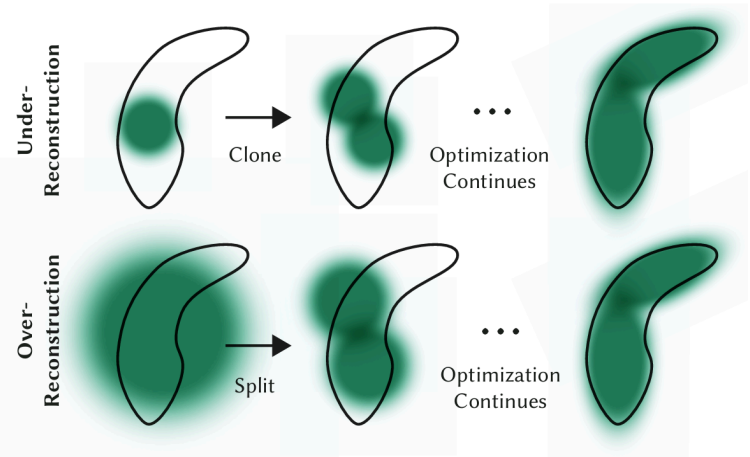


Figure 10: Adaptive Gaussian Densification Scheme [2]

The 3DGS open-source code repository based on the original paper *3D Gaussian Splatting for Real-Time Radiance Field Rendering* by Bernhard Kerbl et al. is available at https://github.com/graphdeco-inria/gaussian-splatting. Another open-source software package we considered is called gsplat, which features a PyTorch-based library for CUDA accelerated computation on NVIDIA GPUs [5]. This repository is accompanied with the paper *gsplat: An Open-Source Library for Gaussian Splatting* by Vickie Ye et al. which is available at https://github.com/nerfstudio-project/gsplat.

For the purposes of compatibility with our personal machines, we utilized OpenSplat as the main library to execute Gaussian splatting in our pipeline for the project. OpenSplat is a C++ implementation that emphasizes portability, compatible with NVIDIA, AMD, and Apple GPUs, as well as CPUs. The repository is available at https://github.com/pierotofy/opensplat. The opensplat.cpp file contains the main training loop including the major components of the Gaussian splatting pipeline discussed. This is where changes to the training loop can be implemented (e.g. manipulating scale every set number of iterations, changing opacity, etc.). In addition, relevant metrics such as the loss function values per iteration and number of gaussians can be extracted here. Another library we've looked into and have added as an option alongside OpenSplat is Faster-GS which implements improved optimization methods for faster performance. The accompanying paper for this repository is *Faster-GS: Analyzing and Improving Gaussian Splatting Optimization* by Florian Hahlbohm et al, with the code available at https://github.com/nerficg-project/faster-gaussian-splatting.

Rasterization. Rasterization is the fast rendering method that transforms a 3D scene into a 2D image and determines how to color each individual pixel. The logic behind rasterization is shared in both the contexts of Gaussian training and the final rendering used for 3D viewing. To achieve real-time rendering for the Gaussian splats, a differentiable tile-based rasterizer was designed. This method splits the screen in 16x16 tiles and culls (or selects) the 3D Gaussians that intersect the view's frustum (or the 3D volume that encases the visible region) [2].

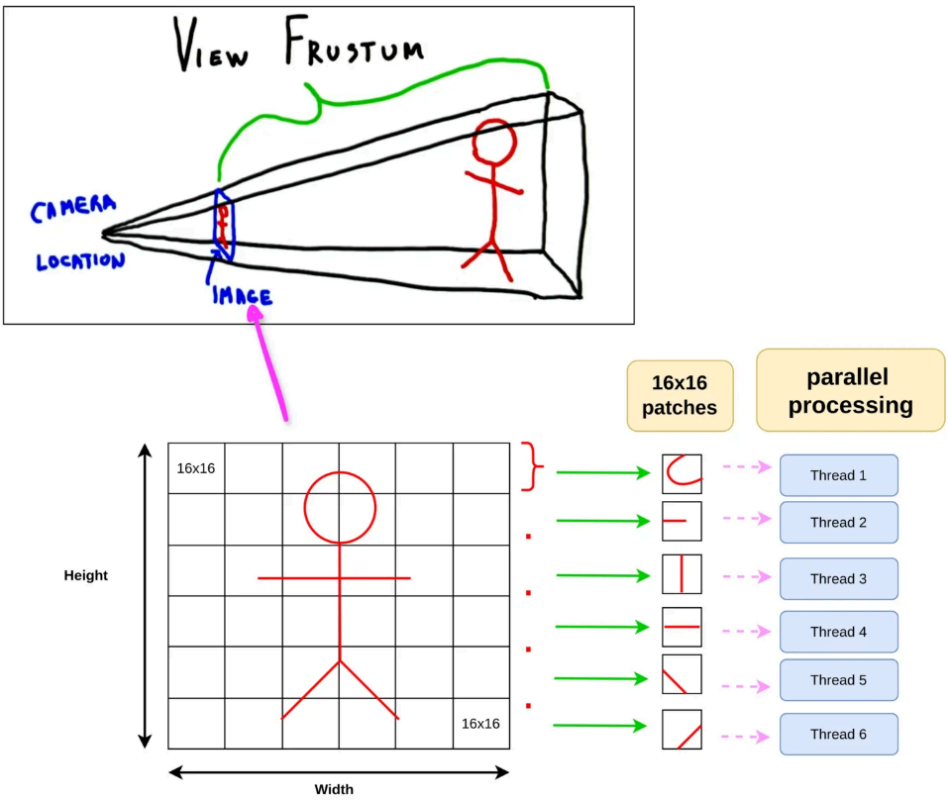


Figure 11: Tile-based Rasterization Pipeline [12]

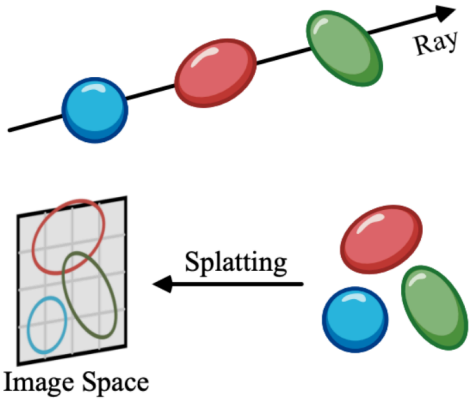


Figure 12. Tile-Based Rendering Splatting [6]

The remaining Gaussians are sorted by depth using an optimized GPU Radix sort. This is to ensure the correct ordering of color mixing from the point-of-view of the camera's position. The list of sorted Gaussians are projected or splatted to each of the tiles and each tile is assigned a thread to perform parallel rendering [2, 6]. After splatting, α-blending is performed to combine the Gaussians and determine their blended color. Using a point-based approach, the color C of a pixel is computed by blending N ordered points overlapping the pixel (Equation 5). The thread stops when a target saturation of α in a pixel is reached. The benefits of the differentiability of the rasterizer, optimizations, and parallel computation culminate in a successful real-time rendering solution.

$$C = \sum_{i \in N} c_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j)$$

Equation 5: Alpha Blending Formula

One more important detail to cover is how color is represented. Each Gaussian has a color c represented by spherical harmonic coefficients. Typical 3D scenes can have pixels that change their color depending on the relative angle the viewer is observing it (e.g. shiny surfaces). Gaussians with a static color, regardless of the viewing angle, would result in a flat, unnatural looking scene. Spherical harmonics are what enable directional lighting, that is they allow Gaussians to change their hue in accordance with the position of the camera. By compiling a sum of basis functions via Fourier series in the spherical dimension, spherical harmonics are able to encode the color as a function of the viewing direction (Figure 13).

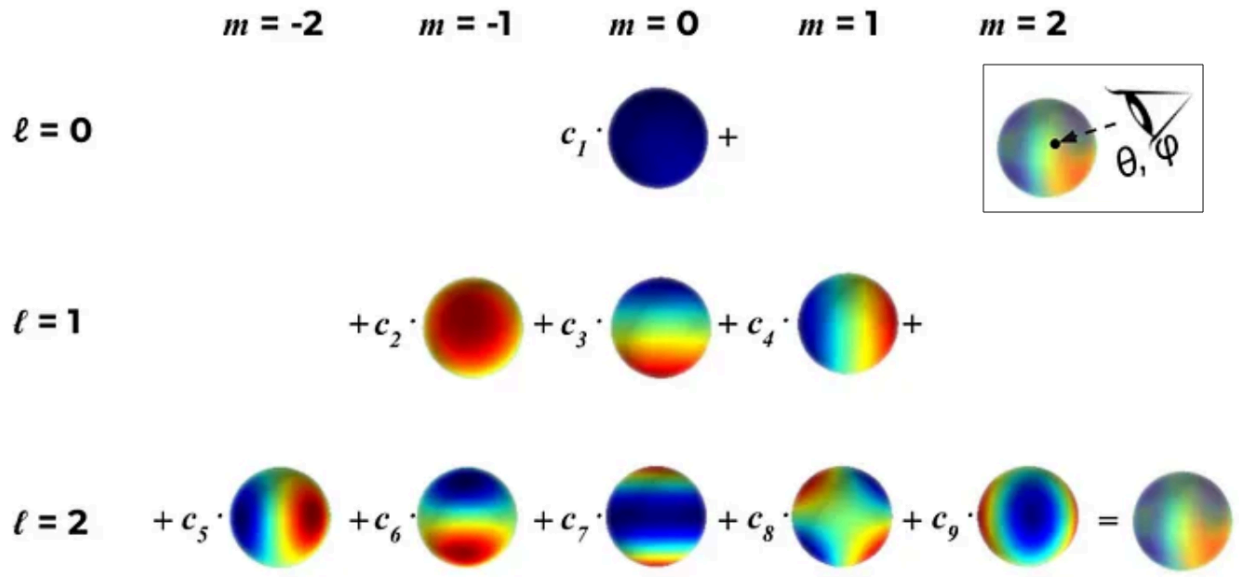


Figure 13: Spherical Harmonics [12]

Due to our project design choice to create a working pipeline in-browser, the 3D viewer we used for our web application is an abstraction of the “splat” repository, available at https://github.com/antimatter15/splat. This renderer differs from the standard CUDA implementation due to web browser security measures, resulting in lower (but workable)

performance. Rather than using tile-based rasterization, this implementation represents Gaussians as quadrilateral bounding boxes that are handled by a vertex shader which performs projection. Sorting, which is normally handled by the GPU and Radix Sort, is now handled by the CPU using a standard CPU-capable sorting algorithm like quicksort or mergesort. Spherical harmonics are also sacrificed for performance (removed for basic RGB). The tradeoff in performance is compatibility with most machines, including PCs without NVIDIA graphics cards, Macbooks, and mobile devices. Figure 14 shows a complete rendering of a University of Florida gator statue utilizing our Gaussian splatting pipeline.

Figure 14: University of Florida Gator Statue - Gaussian splat rendering

II. App User Guide

i. Landing Page

When you first launch the application you will be on the landing page. Which shows an overview of the Gaussian Splatting project. We have a top navigation you can use to go through each page of the application. Each page represents a different view of the application. To start you can click the “Try the Live Demo” button.

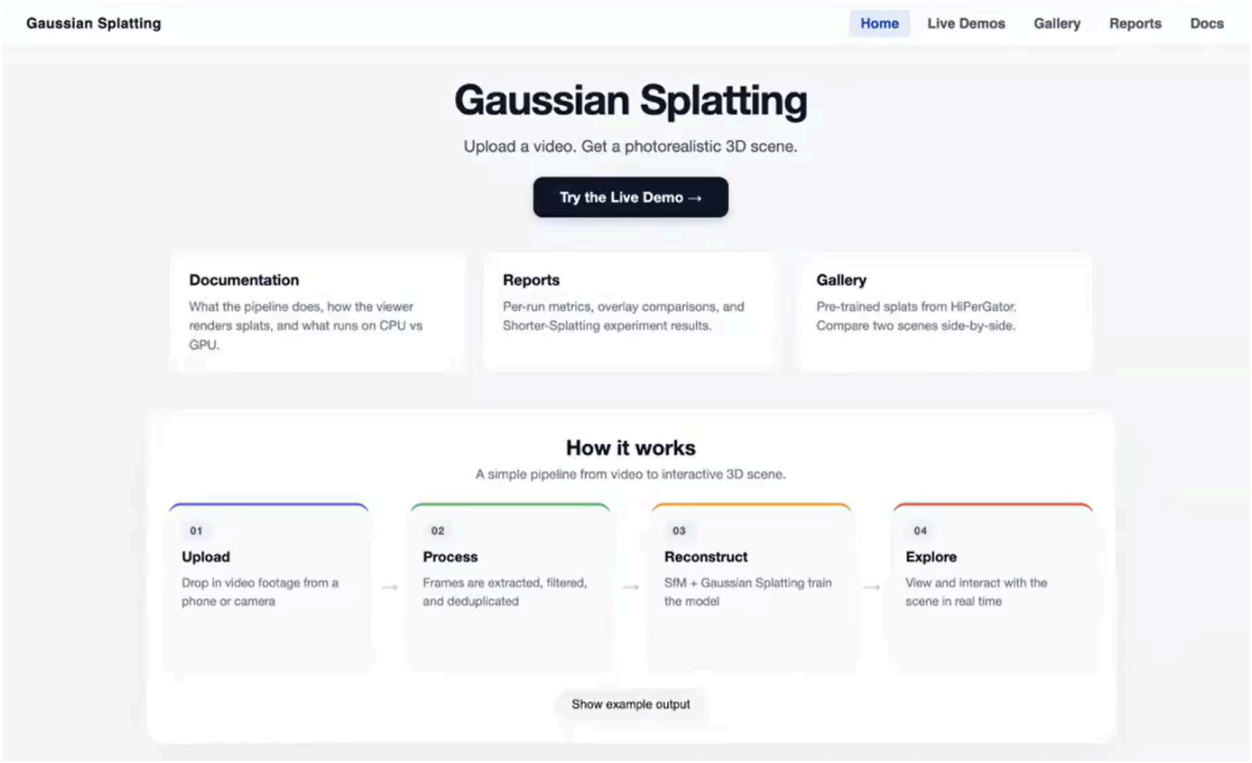


Figure 15: Landing Page with Navigation Menu

ii. Live Demo Page and Uploading

Now you should be on the Live Demo Page. On this page you will upload a video and start running the pipeline. First type a dataset name then choose the video file on your computer. Then can choose a backend process. If you have setup both processes in setup you choose either

one. Then you can click the “Upload Video” button. Next you can either choose to run simple or choose some advanced settings. Can click “Show Advanced” to view advanced settings.

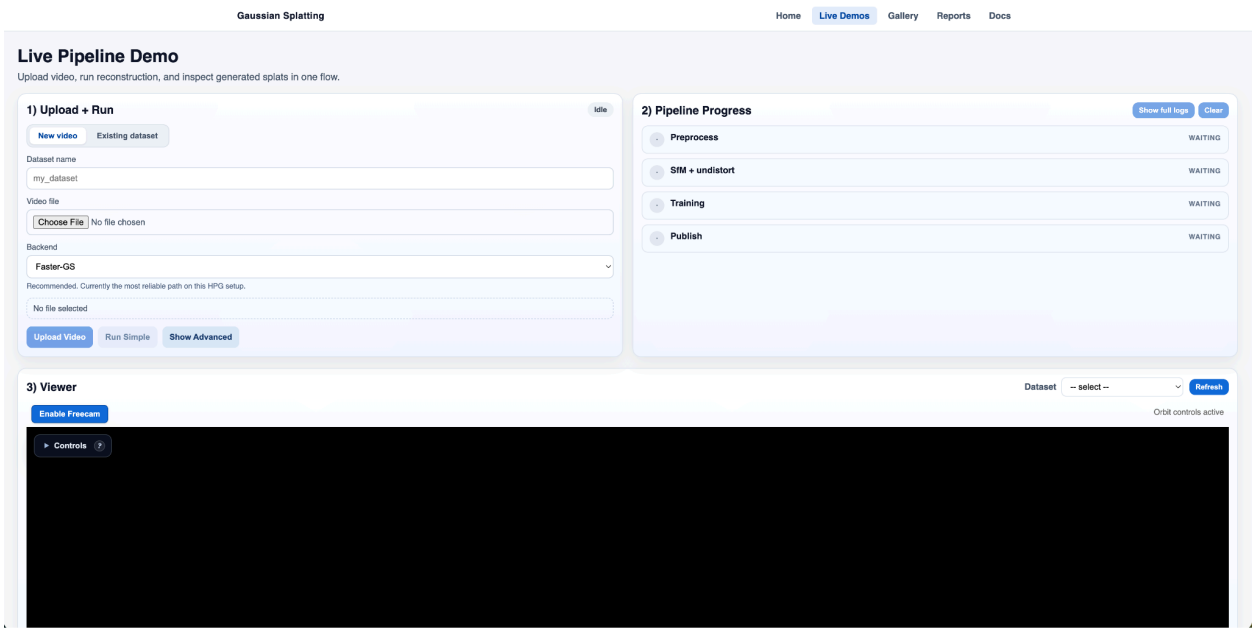


Figure 16: Live Demo Page with ability to run video through pipeline

iii. Advanced Settings

Now that you have the advanced settings open. You have a variety of options to choose from. These advanced settings allow you to tune how you train the Gaussian splatting. These are great for doing research. The available settings are choosing SfM method, extraction FPS, Downscale factor, Max output width, Blur threshold, Duplicate threshold, Iterations, Seed, and shorter splatting experiments. But for now you can just click the “Run Simple” button which has a default of settings.

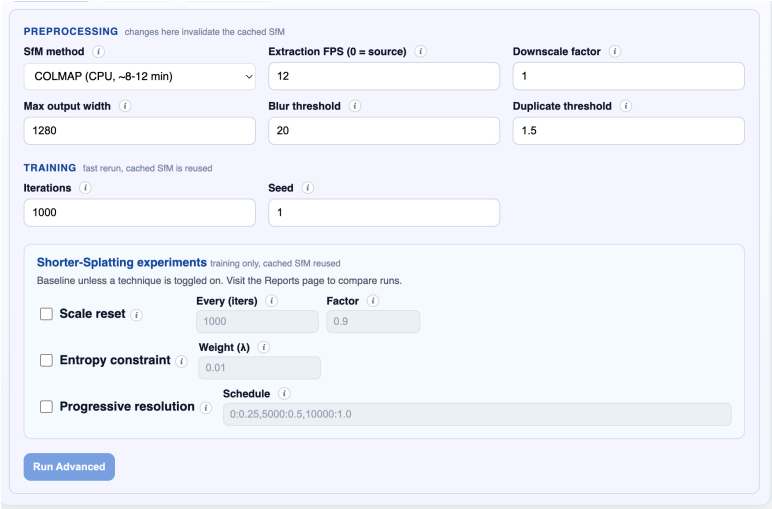


Figure 17: Advanced Control Settings

iv. Viewing 3D Scene

It can take awhile for the video to go through the pipeline. Once it is done it will automatically show the 3D scene in the viewer on the bottom of the page. You can use the camera controls or use the freecam to view the 3D scene.

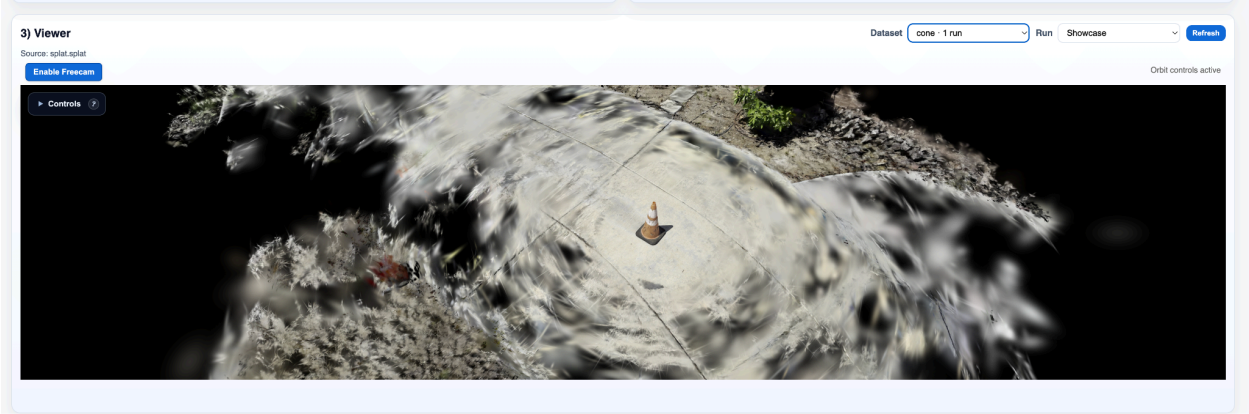


Figure 18: 3D Scene Viewer on Live Demo Page

v. Gallery Page

Now that you have seen the Live Demo and how it works we will move on to the gallery. This is where you can view all the 3D scenes you have rendered through the pipeline. To start all you need to do is click select a dataset dropdown to view all the scenes. Then you can go through each one and even compare two side by side if you ran two on the same video.

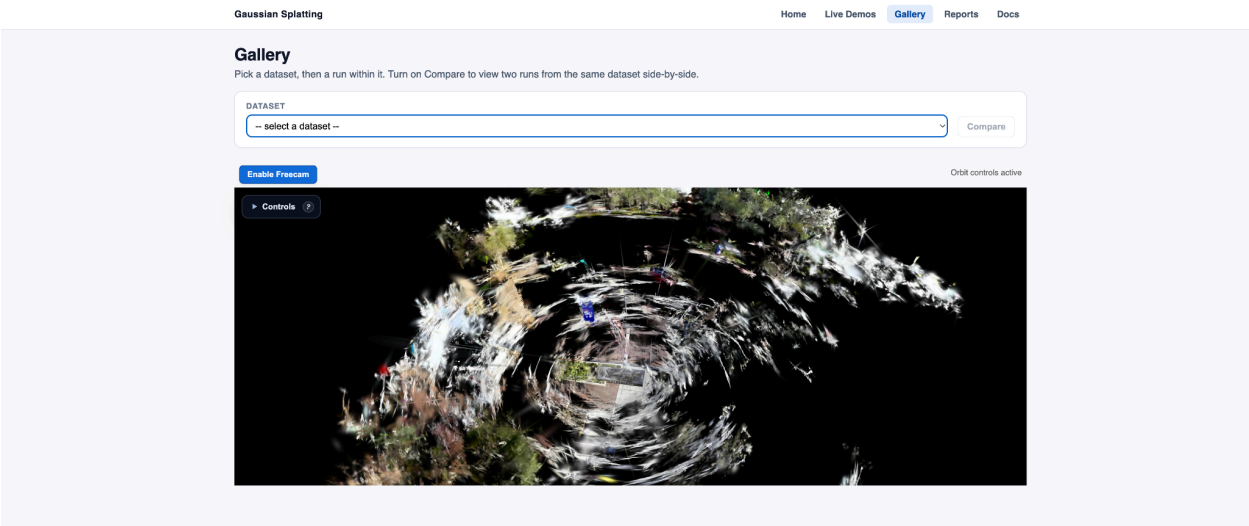


Figure 19: Gallery Page with Ability to Compare two 3D Scenes

vi. Reports Page

To move to the reports page click on the navigation button saying “Reports” and the reports page will load. Here you can view all the metrics on the 3D scene for analysis. You have the ability to view PSNR, Loss, Gaussian count, Wall time, and model distributions. These metrics allow you to quickly assess how the 3D render did and if it is a high quality reconstruction.

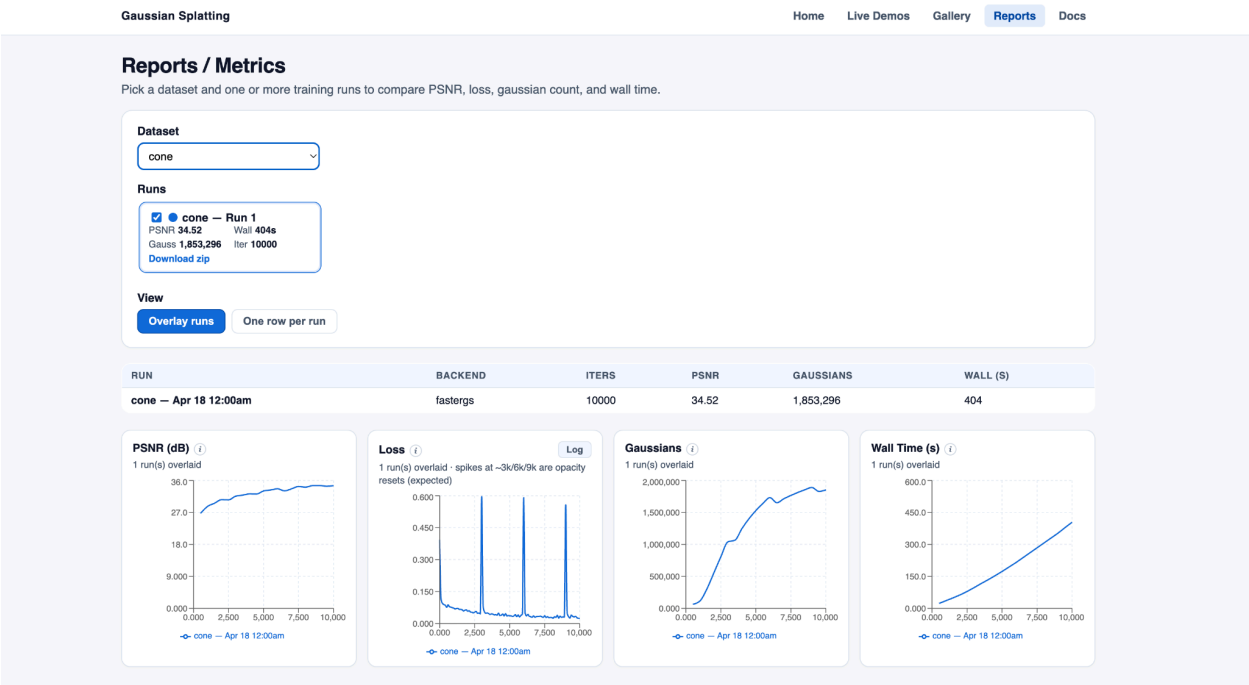


Figure 20: Reports Page showing PSNR, Loss, Gaussians count, and Wall time

When you scroll down a little bit you can see the PSNR vs Wall time and model distributions.

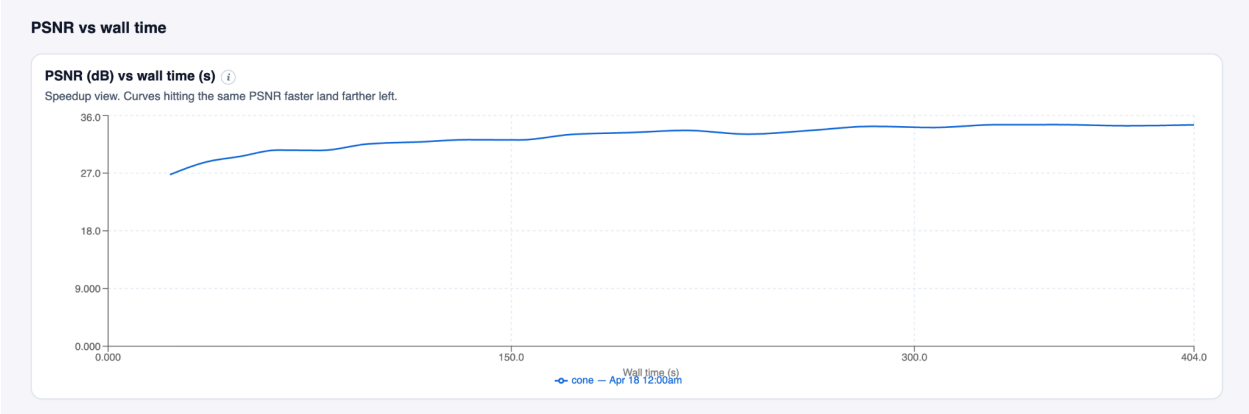


Figure 21: PSNR vs Wall time Graph

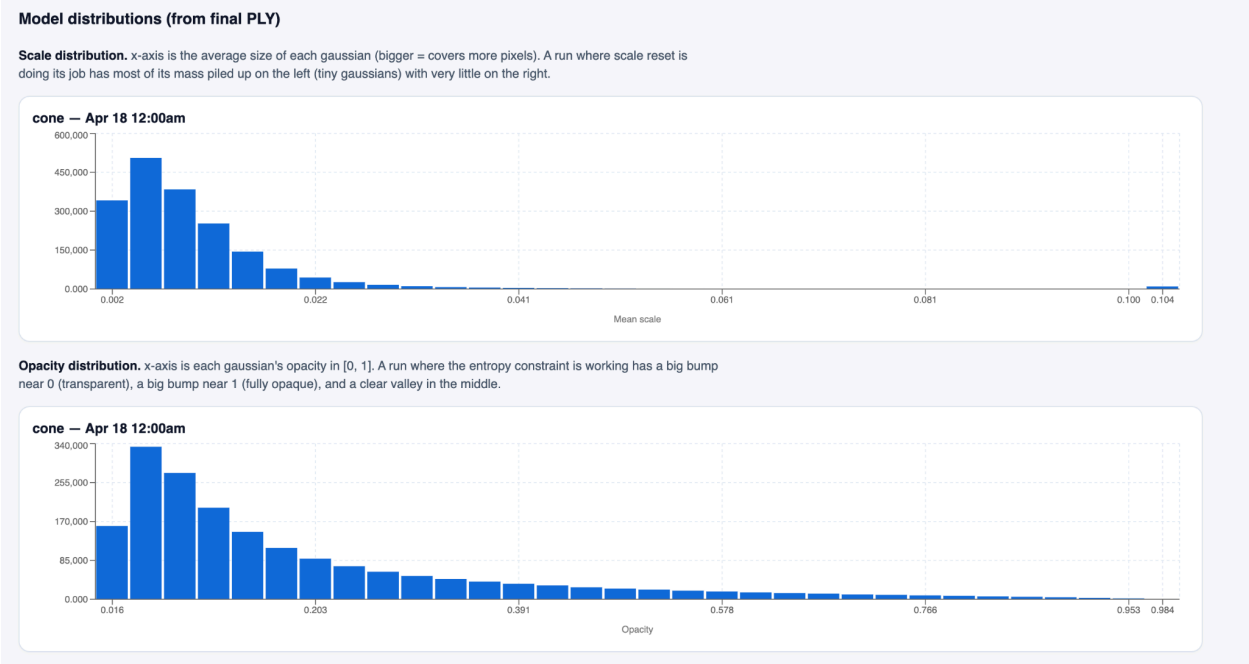


Figure 22: Model Distributions including Scale and Opacity Histograms

Transition Plan/Next Steps

If we furthered our project, the focus would shift from developing our application to using it as a research tool. Since we created an end-to-end gaussian splatting pipeline, we have

the ability to test different approaches as they are suggested in scientific literature. Thus, we would be able to experiment with different techniques, making real contributions to the field and improving the gaussian splatting process.

An optimization paper we're looking into implementing is *Speeding Up the Learning of 3D Gaussians with Much Shorter Gaussian Lists* by Jiaqi Liu and Zhizhong Han. The paper claims that it could speed up the Gaussian splatting process by shortening each Gaussian list used to render a pixel [9]. Specifically, they shrink the size of each Gaussian by resetting their scales on a set period, inciting smaller Gaussians to cover fewer nearby pixels, shortening the Gaussian lists of pixels. Allegedly this is able to speed up performance without sacrificing rendering quality. Additionally, they implement an entropy constraint on the alpha blending procedure to drive dominant weights large while reducing the impact on nearby pixels, further shortening lists. They also integrate a rendering resolution scheduler to further improve efficiency through progressive resolution increase. We've already begun to implement these solutions into our Faster-GS library, but further work is needed to properly verify our implementation and compile the results for accurate analysis.

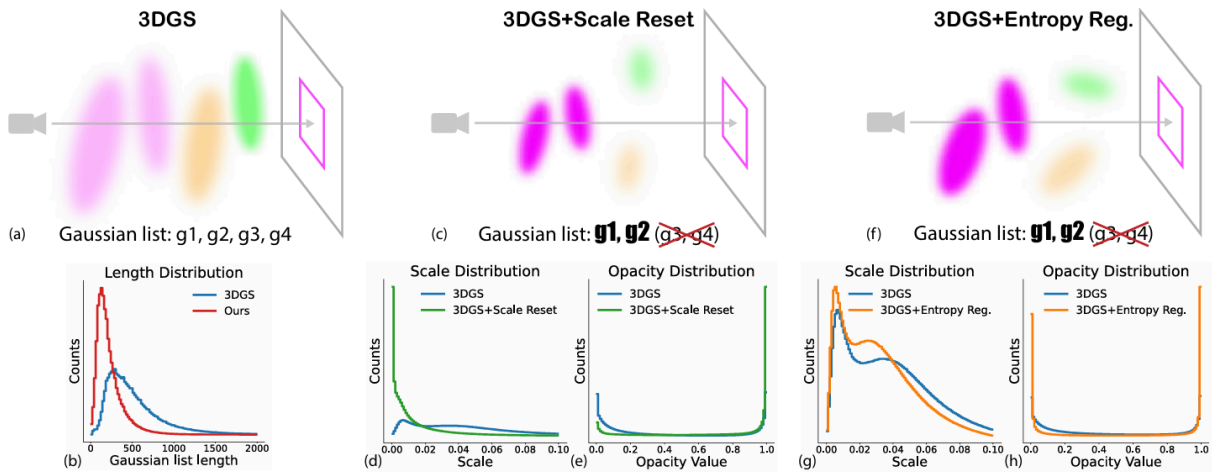


Figure 23: Overview of Shorter List Optimizations. (a) 3DGS Gaussian list. (b) Distribution of Gaussian list length. (c) Gaussian list reduction after applying scale reset. (d, e) Scale and opacity distribution with scale reset. (f) Gaussian list reduction after applying entropy regularization. (g, h) Scale and opacity distribution with entropy regularization. [9]

Acknowledgements

We would like to thank Professor Entezari for serving as our project advisor and for his guidance and support throughout the project. Additionally we would like to thank Professor Thomas and the CIS4914 TA's for facilitating the senior project course. Finally, we are very thankful to the UF Research Computing team for access to HiPerGator resources, greatly increasing the power of testing as well as cross compatibility for our team.

Appendices

Appendix A: 3DGS Gradient Descent (gsplat implementation)

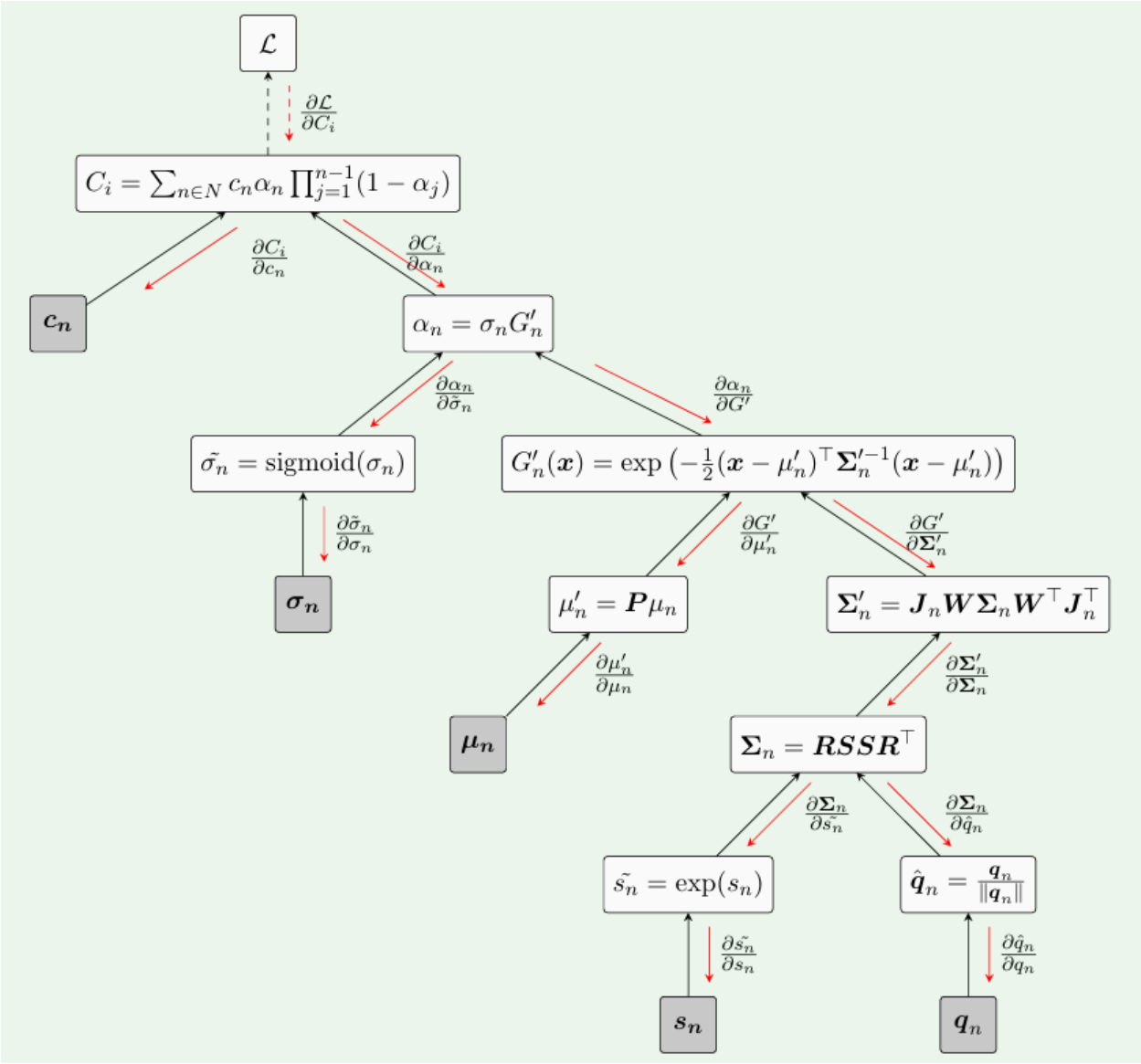


Figure 24: An illustration of forward and backward computation graphs of the gsplat Gaussian Splatting rendering function. [5]

Biography & Individual Contributions

Jackson Culbreth

Biography:

I am graduating from the University of Florida in May 2026 with a Bachelor of Science in Computer Science and minors in Statistics and Electrical Engineering. Throughout my time at UF I have been part of a UAV (unmanned aerial vehicle) design and building club that has given me a strong interest in embedded systems and mission-critical systems. In July, I will start a full time job with Lockheed Martin as an Embedded Software Engineer. In my free time I am very active with my church community with a focus on discipling younger men and building leadership skills.

Contributions:

As team lead, I began the Github initialization process and got our initial demo of Structure from Motion functional with the use of a public dataset. As we got closer to Spring Break, HiPerGator access was still not granted. I had previously considered this a blocker to even starting the Gaussian Splatting portion of our project, however in weeks 7-8 I found a C++ based implementation called OpenSplat that was able to run on my machine. This allowed me to run gaussian splats of small datasets on my laptop. I also departed from the sole use of online public datasets, and collected a video of a small bluetooth speaker to test the system.

After Spring Break, our whole team worked together to prepare some reports on performance metrics that Dr. Entezari requested. In particular, I gathered data on the SfM process to pass on to Josh. As we reached the final stretch, I overhauled our frontend to split the previous

single-page model into a landing page and pages for each component of our project. I created the first version of our live demos page, allowing the user to interact more fully with the backend pipeline. Finally, I collected some videos on campus and passed them to Nick to process on HiperGator. These are the UF themed renders you saw in our demo.

Prior to this project, I had briefly heard from some lab members that there was a new graphics technique called Gaussian Splatting, but I did not look into it myself and considered it rather complex and beyond my scope of understanding. Taking Dr. Entezari's special topics course on graphics awakened my interest in many graphics-related topics, and I was very intrigued by the different ways computers represent and display geometry. Having a good understanding of triangle meshes, it's easy to come to the conclusion that you generally need many millions of triangles, a good shading program, or both to get a high quality photorealistic scene that can be rendered. This is the amazing part about Gaussian Splatting: While it does not always guarantee the same geometric accuracy as a triangle mesh, the use of overlapping fuzzy blobs (gaussians) gives amazing levels of detail. At the core of it all, it really clicked for me that instead of brute forcing a realistic scene into more and more polygons, we can embrace the "fuzziness" of the world around us and use a different approach.

Joshua Bowman

Biography:

I'm graduating from the University of Florida in May 2026 with a degree in Computer Science. I currently work at a Software Company as a BI Reporting Developer, where I build

dashboards and reports that help the executives make sense of their data. Outside of school and work, I'm a professional halfpipe snowboarder. I've competed at the X Games twice and was a 2022 Olympic alternate. Snowboarding has taught me a lot about staying focused under pressure and trusting the reps, which shows up in how I approach both work and school.

Contributions:

I served as Scrum Master for this project, which involved planning and coordinating with the team. My technical contributions were mostly on infrastructure and validation testing. I built out the Faster-GS integration with HiPerGator, which included SLURM dispatch, SfM caching, and conda environment bootstrap. This whole process was difficult to build because I had to learn what can run on each GPU. I ran into some limitations where the FasterGS rasterizer was built for an A100 GPU architecture and HiPerGator only has L4 and B200. So different kinds of shared memory than the A100. So I had to go back to their stock Inria rasterizer instead of custom built. Which is a bummer because we lost performance.

I also helped build out part of the frontend so it shows what's happening when you upload a video, and I added full setup documentation for running our project on your own computer. I also built out the Reports page, which automatically ingests data when you run a dataset, using the metrics that Jackson and I had built to validate our pipeline. Finally, I ran a huge final code audit and cleanup which involved going through a whole repository to see what was missing and code not needed anymore. I also helped get the setup and readme documentation to read cleaner and simple. To end it off I made it possible so you can see a sample version of the application

through GitHub Pages.

During the whole project I learned a ton on Gaussian splatting. I did not know about it before this project so I had to do a lot of research to learn about it. After researching I was able to find some newer sources on the Gaussian Splatting and integrate them into our pipeline. I really wanted to be able to do some cool research with this project and anyone else in the future that wants to use it. I have had a lot of experience before this project doing data analysis. So I was able to use that knowledge to build out our reporting and do validation on the results. Using this knowledge I was able to turn the messy data into ingestible graphs the whole team could use to show how our datasets were doing.

Jackson Kelly

Biography:

I am graduating this semester with a B.S. in Computer Science and a minor in statistics. I have previously interned at the Federal Reserve Board and United Parcel Service working in full-stack development. After graduation I intend to pursue a career as a software developer.

Contributions:

Throughout this project, I primarily focused on backend development. At the start, I was responsible for developing the video ingestion and preprocessing steps of the pipeline. I implemented the API for video handling, and worked on various preprocessing techniques to ultimately improve the optimization performance. I then worked on connecting all the modular components of the pipeline in order to have a true end-to-end application. This consisted of refactoring in order to ensure that a user could upload on the frontend, and all other components

were sufficiently coordinated to provide a final render at the conclusion of the pipeline. Additionally, this required work to ensure compatibility across operating systems for our pipeline, as some development was done on Mac and Windows. After that, I worked on the advanced controls for our pipeline. This allowed a user to have increased control over the gaussian splatting from the application's frontend, ultimately improving the application's ability to be used as a research tool. Finally, I worked on the frontend UI, primarily on designing the landing page in an intuitive way that introduced our project while ensuring it was still easy to navigate the site.

Before starting this project, I had no experience working with computer graphics. This project required me to learn the basics in order to understand the complexities of what we were trying to accomplish. When we first started development, I believed I had a fairly solid conceptual understanding of the topic, but throughout development, research, and presentations I gained a much more significant depth of knowledge. Development taught me how a full machine learning pipeline fit together, and how we could use individual components, such as Opensplat and OpenSfM, to create a new, unified system. I learned about Structure-from-Motion, estimating camera positions and creating point clouds, and how gaussian splatting worked, initializing small blobs with parameters and optimizing their error, and how it differed from traditional approaches. I gained experience dealing with dependencies and binaries, and the issues that can come from them. Ultimately, the project taught me how to deal with real world systems and improved my skills as a developer. The novelty of the field made the project exciting, as we watched new advancements and approaches be developed while our project was in progress. I found it to be a valuable learning experience and am proud of our end product.

Nicholas Desmornes

Biography

I'm graduating from the University of Florida in Spring 2026 with a B.S. in Computer Science. I've previously worked at the US Army Corps of Engineers and Wells Fargo as a software developer. My goal is to pursue a career in web development after graduation.

Contributions:

My main contributions to this project involved assistance with implementation on frontend elements and literature review. I've implemented the Gaussian splatting 3D viewer as well as some elements to help upload datasets in the early stages of the project. After our first advisor meeting I helped with the compilation and visualization of important metrics in the Gaussian training pipeline including the loss function and Gaussian count per iteration, as well as a roadmap of future metrics to compile and analyze such as PSNR and computation time. In addition, I looked into researching recent papers that utilized newer optimization methods to try implementing in our solution. For the senior showcase, I helped run input videos through our Gaussian splatting pipeline on HiperGator to populate the gallery, as well as prepare an acceptable video to use for our live demo. Further work included experimenting with varying input parameters, images, and optimization techniques.

Throughout this project, I learned the most through rigorous research and seeking learning materials to help supplement my understanding of the details. I was able to gain deeper knowledge of the mathematics behind the major steps of the overall Gaussian splatting pipeline,

from SfM to rendering. Studying references also helped me appreciate that Gaussian Splatting didn't procure itself spontaneously, but was rather years of academic work and research building off of one another making progressions in the field. Ultimately, this project helped me become a better research student and gave me a taste of what it takes to pursue a career in academia.

References

[1] B. Mildenhall, P. Srinivasan, M. Tancik, J. Barron, R. Ramamoorthi, and R. Ng, "NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis," Aug. 2020. Available: https://arxiv.org/abs/2003.08934

[2] Bernhard Kerbl, Georgios Kopanas, T. Leimkühler, and G. Drettakis, "3D Gaussian Splatting for Real-Time Radiance Field Rendering," ACM Transactions on Graphics, vol. 42, no. 4, pp. 1–14, Jul. 2023, doi: https://doi.org/10.1145/3592433.

[3] J. Schönberger and J.-M. Frahm, "Structure-from-Motion Revisited," 2016. Available: https://openaccess.thecvf.com/content_cvpr_2016/papers/Schonberger_Structure-From-Motion_Revisited_CVPR_2016_paper.pdf

[4] R. Hartley and A. Zisserman, Multiple View Geometry in Computer Vision, 2nd ed. Cambridge, UK: Cambridge University Press, 2004.

[5] V. Ye et al., “gsplat: An Open-Source Library for Gaussian Splatting,” arXiv.org, 2024. https://arxiv.org/abs/2409.06765

[6] G. Chen and W. Wang, “A Survey on 3D Gaussian Splatting,” arXiv (Cornell University), Jan. 2024, doi: https://arxiv.org/abs/2401.03890

[7] F. Hahlbohm, L. Franke, M. Eisemann, and M. Magnor, “Faster-GS: Analyzing and Improving Gaussian Splatting Optimization,” Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), June 2026. arXiv:2602.09999. Available: https://arxiv.org/abs/2602.09999

[8] A. Hanson, A. Tu, G. Lin, V. Singla, M. Zwicker, and T. Goldstein, “Speedy-Splat: Fast 3D Gaussian Splatting with Sparse Pixels and Sparse Primitives,” Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 21537–21546, June 2025. Available: https://speedysplat.github.io/

[9] J. Liu and Z. Han, “Speeding Up the Learning of 3D Gaussians with Much Shorter Gaussian Lists,” arXiv.org, 2026. https://arxiv.org/abs/2603.09277

[10] “3D Gaussian Splatting,” Mashaan blog, 2024. https://mashaan14.github.io/YouTube-channel/nerf/2024_10_14_3DGS

[11] Jaykumaran, “MASt3R and MASt3R-SfM Explanation: Image Matching and 3D Reconstruction Results,” LearnOpenCV – Learn OpenCV, PyTorch, Keras, Tensorflow with code, & tutorials, Mar. 25, 2025. https://learnopencv.com/mast3r-sfm-grounding-image-matching-3d/

[12] soumyadip, “3D Gaussian Splatting Introduction - Paper Explanation & Training on Custom Datasets with NeRF Studio Gsplats,” LearnOpenCV – Learn OpenCV, PyTorch, Keras, Tensorflow with code, & tutorials, Dec. 17, 2024. https://learnopencv.com/3d-gaussian-splatting/#Introduction-to-3D-Gaussian-Splatting